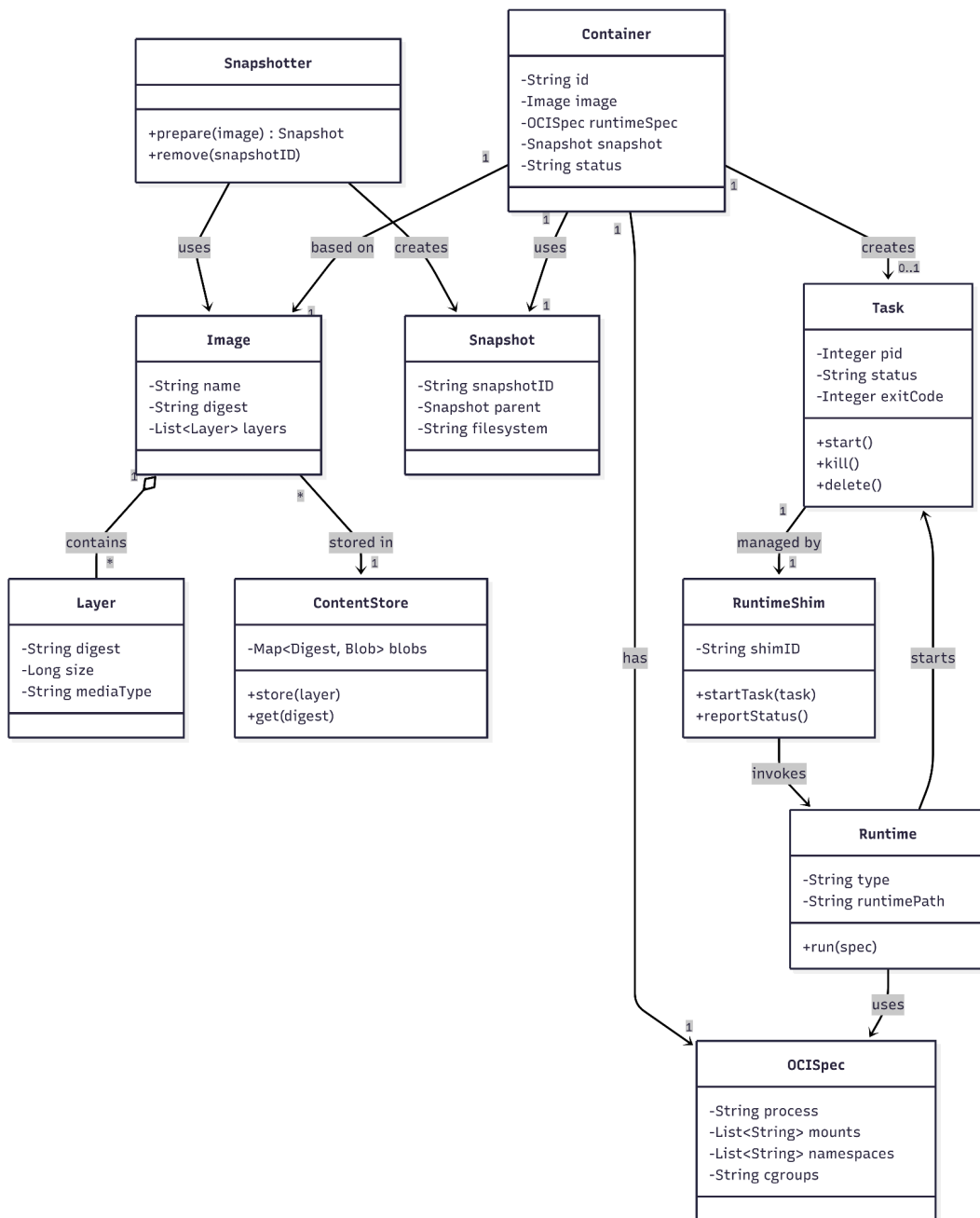




1. Class Diagram: containerd 的靜態結構

這張圖是在說明 **containerd** 內部有哪些核心物件以及它們之間的關係。

containerd 在管理一個容器時，不只是單純「執行一個程式」，而是需要處理很多東西，例如 image、image layer、filesystem、container 設定、實際執行中的 process 等等。

Image 代表容器映像檔，例如 nginx、ubuntu。Image 裡面包含多個 **Layer**，這些 Layer 是組成容器檔案系統的基礎。這些 Layer 會被存在 **ContentStore** 裡面，所以 ContentStore 可以理解成 containerd 儲存 image 內容的地方。

接著, **Snapshotter** 會根據 **Image** 建立 **Snapshot**。Snapshot 的作用是準備 container 執行時需要的 root filesystem。也就是說, Image 本身只是映像檔資料, 而 Snapshot 是讓 container 真正可以使用的檔案系統狀態。

Container 則是容器的靜態定義。它包含 container id、使用的 image、runtime spec、snapshot 和 status。這裡要特別注意的是, Container 並不等於正在執行的程式。Container 比較像是「這個容器應該怎麼被執行」的設定資料。

真正執行中的容器 process 是 **Task**。Task 裡面有 pid、status、exitCode, 也有 start、kill、delete 這些操作。這表示 containerd 把 **Container** 和 **Task** 分開設計：

None

Container = 容器的靜態設定與 metadata

Task = 實際執行中的 container process

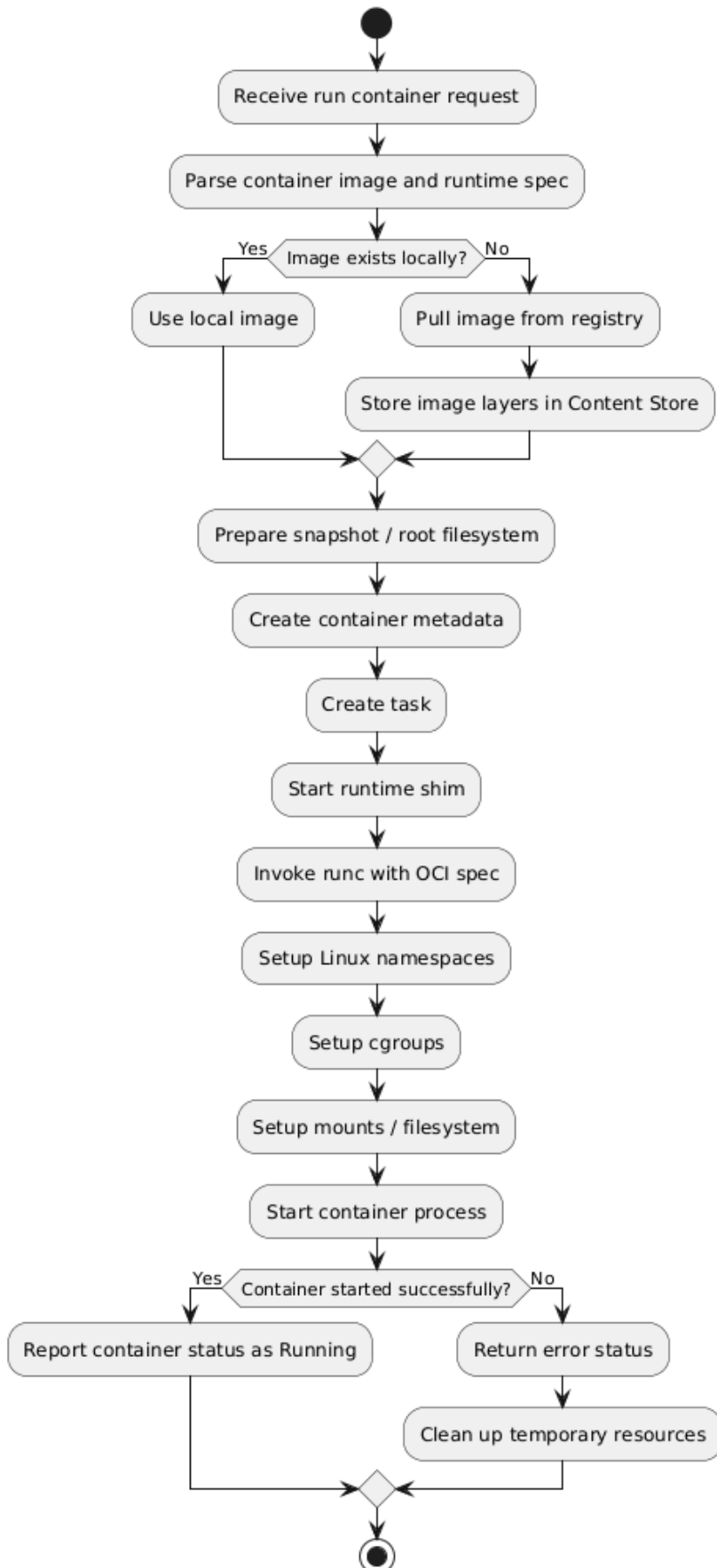
這樣設計的好處是, containerd 可以清楚區分「容器的定義」和「容器的執行狀態」。例如一個 container 可以先被建立, 但還沒有啟動; 等到需要執行時, 才建立 Task。

最後, Task 會由 **RuntimeShim** 管理, RuntimeShim 再去呼叫 **Runtime**, 例如 runc。
Runtime 會根據 **OCISpec** 啟動真正的容器程序。OCISpec 裡面包含 process、mounts、namespaces、cgroups 等資訊, 也就是底層容器隔離需要的設定。

containerd 的設計重點：

containerd 把容器執行過程拆成多個清楚的物件：**Image** 負責映像檔, **ContentStore** 負責儲存內容, **Snapshot** 負責檔案系統, **Container** 負責靜態設定, **Task** 負責實際執行, **RuntimeShim** 和 **Runtime** 負責接到最底層的容器執行環境。這種分工讓 containerd 不會把所有事情混在一起, 而是可以模組化地管理 container 的生命週期。

containerd - Activity Diagram for Starting a Container



2. Activity Diagram: containerd 啟動容器的整體流程

containerd 從收到啟動請求, 到 **container** 成功執行的完整流程。

流程開始: **Receive run container request**, 也就是 containerd 收到上層系統的請求。這個請求可能來自 Docker, 也可能來自 Kubernetes。上層系統只會說:「我要執行某個 image」, 但底層細節會交給 containerd 處理。

接著 containerd 會解析 container image 和 runtime spec。runtime spec 可以理解成容器要怎麼被執行的設定, 例如要跑哪個 process、要掛載哪些檔案系統、要使用哪些 namespace 和 cgroup。

之後 containerd 會判斷 **Image exists locally?**, 也就是本機是否已經有這個 image。

如果本機沒有 image, containerd 會: **Pull image from registry -> Store image layers in Content Store** 也就是從 registry 把 image 下載下來, 並把 image layers 存進 ContentStore。(如果本機已經有 image, 就可以直接使用 local image)

不管 image 是新下載的, 還是本來就存在, 接下來都會進入 **Prepare snapshot / root filesystem**, 這一步是透過 Snapshotter 準備 container 執行需要的檔案系統。因為 container 不能直接拿 image layer 來跑, 它需要一個整理好的 root filesystem。

接著 containerd 會建立: **Create container metadata -> Create task**
container metadata 是容器的靜態資料, 例如 id、image、snapshot、runtime spec。Task 則代表真正要被啟動的 container process。

接下來 containerd 會: **Start runtime shim -> Invoke runc with OCI spec**
Runtime shim 是 containerd 和 runc 中間的橋樑。containerd 不會直接讓自己跟每個 container process 綁死, 而是透過 shim 管理每個 task。這樣即使 containerd daemon 本身重啟, 正在執行的 container 也比較不會被直接影響。

然後 runc 會依照 OCI spec 設定底層 Linux 機制:

Setup Linux namespaces

Setup cgroups

Setup mounts / filesystem

這幾個是 container 隔離的核心:

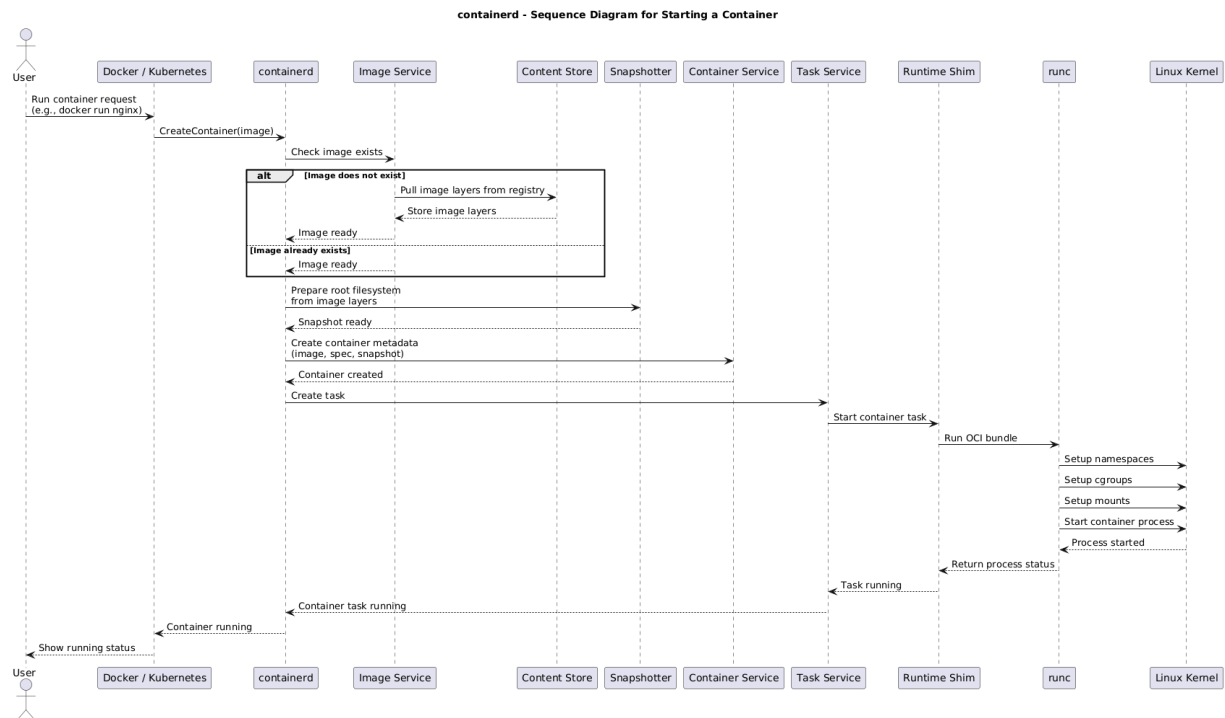
namespaces: 隔離 process、network、mount 等資源

cgroups: 限制 CPU、memory 等資源使用

mounts / filesystem: 建立 container 自己看到的檔案系統

最後 runc 啟動 container process。如果成功, containerd 回報狀態為 Running; 如果失敗, 就回傳 error status, 並清理暫時資源。

Activity Diagram 可以用來說明：**containerd** 將啟動容器的流程標準化。它不只是單純呼叫 **runc**，而是先處理 **image**、**content store**、**snapshot**、**container metadata**、**task**，再交給 **runtime** 啟動容器。意即 containerd 解決的是「容器啟動流程太複雜」的問題。它把流程拆成固定步驟，讓 Docker 或 Kubernetes 不用自己重複實作這些底層細節。



3. Sequence Diagram: 啟動 container 時各元件如何互動

說明啟動 **container** 的過程中，各個元件之間的互動順序。Activity Diagram 比較像流程圖，而 Sequence Diagram 比較強調「誰跟誰溝通」。

一開始是 User 發出 request, 例如: `docker run nginx`

這個請求會先到 Docker / Kubernetes, 然後 Docker / Kubernetes 再向 containerd 發出: `CreateContainer(image)`。containerd 收到請求後，第一件事是請 Image Service 檢查 image 是否存在。

如果 image 不存在, Image Service 會向 Content Store 處理 image layers 的儲存。這裡可以理解成 image 會被拆成一層一層的 layer, 下載後放到 Content Store 裡。(對應到activity diagram的分支部分) 如果 image 已經存在, 就直接回傳 image ready。

接著 containerd 會請 Snapshotter: `Prepare root filesystem from image layers`

Snapshotter 會根據 image layers 準備 container 的 root filesystem, 準備好之後回傳 Snapshot ready。然後 containerd 會請 Container Service 建立 container metadata。這一步會把 image、spec、snapshot 等資訊記錄成一個 container 定義。

接著 containerd 會請 Task Service 建立 task。Task Service 再呼叫 Runtime Shim, 讓 Runtime Shim 啟動 container task。Runtime Shim 接著呼叫 runc: **Run OCI bundle**

runc 會開始跟 Linux Kernel 互動, 設定:

- Setup namespaces
- Setup cgroups
- Setup mounts
- Start container process

Linux Kernel 完成 process 啟動後, 狀態會一路回傳, 最後使用者看到 container running。



這張圖要強調的重點

Sequence Diagram 說明, **containerd** 是中間協調者。它自己不是單獨完成所有工作, 而是協調 **Image Service**、**Content Store**、**Snapshotter**、**Container Service**、**Task Service**、**Runtime Shim**、**runc** 和 **Linux Kernel**。也就是說, containerd 的核心價值不是「單純啟動 process」而是把一整套容器執行流程包成穩定的服務, 提供給 Docker 和 Kubernetes 使用。

對話連結參考: <https://chatgpt.com/share/69f5cb8f-9d94-83a4-8e97-537bdd6608e9>